

HIGH | CVSS 7.5

Description: The API references user cart objects via sequential integer IDs without validating whether the requesting session owns the target basket. Modifying the basket identifier from 1 to 2 exposes unauthorized user cart contents.

[illegible]

Remediation: Implement server-side verification ensuring `basket.UserId === req.user.id` before returning object records, and utilize non-sequential UUIDs.

SEC-03: DOM-based Cross-Site Scripting (DOM XSS) in Search Filter

MEDIUM | CVSS 6.1

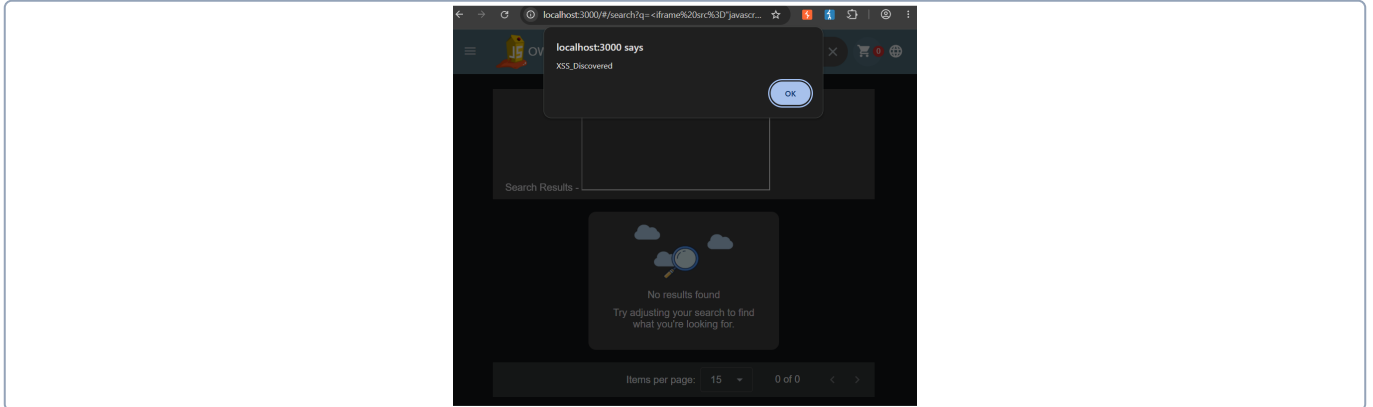
Vulnerability Class: OWASP A03:2021 - Injection (CWE-79) | **Endpoint:** GET /#/search?q=

Description: The search query parameter is directly written to the client-side DOM sink without contextual HTML entity encoding. Injecting an inline iframe payload executes arbitrary client-side JavaScript within the browser context.

Exploit Payload:

```
<iframe src="javascript:alert(`XSS_Discovered`)">
```

Proof of Concept (JavaScript Execution in Client DOM):



Remediation: Enforce strict contextual output encoding (HTML entity escaping) and configure a robust Content Security Policy (CSP) restricting inline script execution.

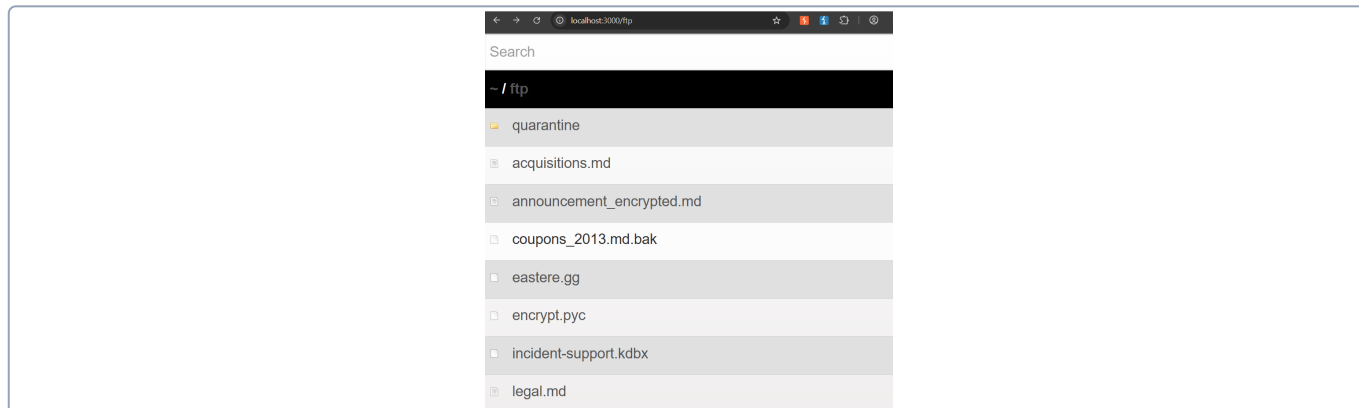
SEC-04: Sensitive Directory Listing & Confidential Asset Exposure

MEDIUM | CVSS 5.3

Vulnerability Class: OWASP A05:2021 - Security Misconfiguration (CWE-548) | **Endpoint:** GET /ftp

Description: The application exposes an open FTP backup directory discovered via /robots.txt. Directory indexing is enabled, permitting unauthenticated visitors to view and download internal backup files, incident support databases (incident-support.kdbx), and confidential acquisition notes (acquisitions.md).

Proof of Concept (Exposed Directory Indexing):



Remediation: Disable web server directory browsing/indexing and restrict public routing to sensitive storage folders using strict access-control filters.

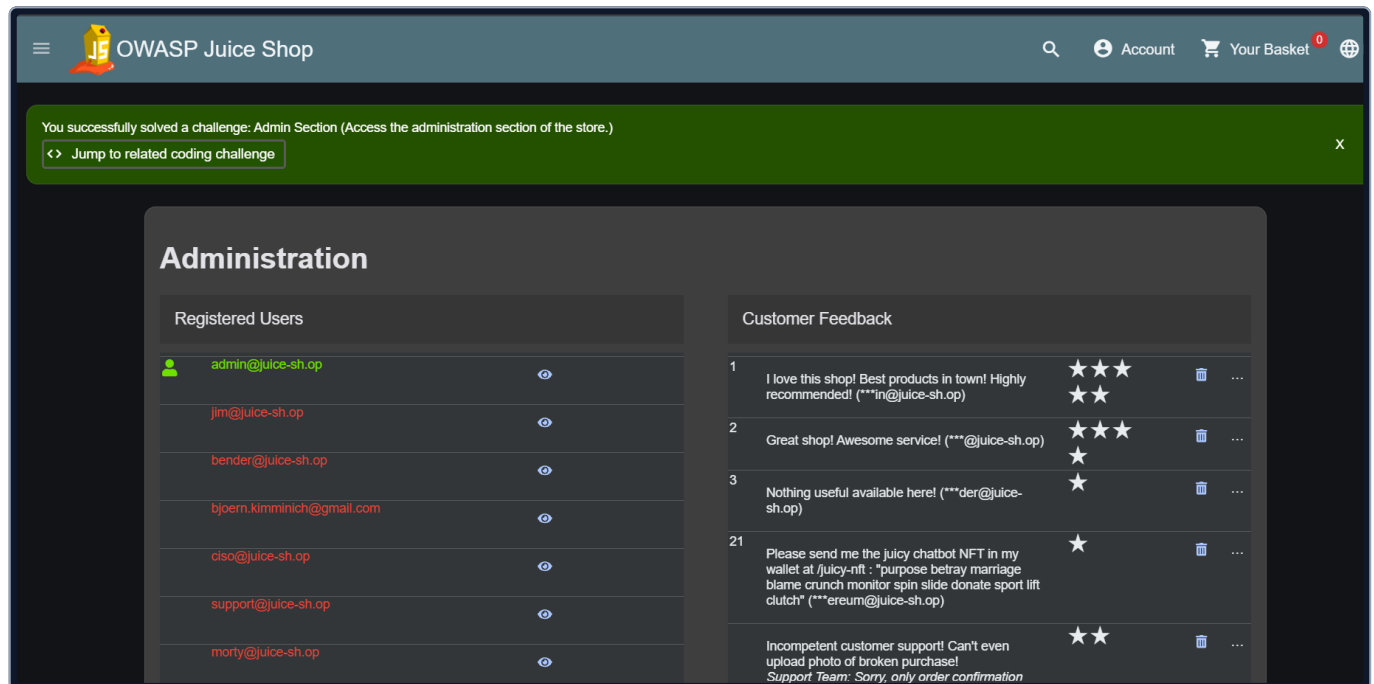
SEC-05: Broken Function Level Authorization (Hidden Admin Dashboard)

HIGH | CVSS 7.5

Vulnerability Class: OWASP A01:2021 - Broken Access Control (CWE-285) | **Endpoint:** GET /#/administration

Description: The application fails to enforce server-side role verification on administrative frontend routes and underlying moderation APIs. By inspecting the frontend JavaScript bundle (main.js), an attacker identifies the client-side route and navigates directly to /#/administration, accessing registered user accounts and customer feedback management tools.

Proof of Concept (Unauthorized Admin Panel Access):



Remediation: Implement Role-Based Access Control (RBAC) middleware verifying user privileges on both frontend route guards and backend API handlers before serving administrative components and data.

4. ASSESSMENT CONCLUSION

The penetration test confirmed significant vulnerabilities across authentication integrity, broken access controls, client-side script execution, and sensitive storage disclosure. Remediation priorities must focus on enforcing parameterized queries, strict role-based object authorization, contextual HTML encoding, and route-level authorization middleware.